

AI 기반 제조현장 작업자 안전 및 위험상황 검출 시스템

김재영*, 안동혁**

AI-based Worker Safety and Hazard Detection System in Manufacturing Sites

Jaeyoung Kim*, Donghyeok An**

요약 본 논문은 제조 및 건설 현장의 안전사고 예방을 위해 라즈베리파이 기반의 AI 작업자 안전 및 위험상황 검출 시스템을 제안한다. 기존 시스템의 낮은 범용성과 작업자/비작업자 구분 불가능 문제를 해결하기 위해 '엣지-서버-웹'의 3-Tier 아키텍처를 설계하고, 작업복 및 안전모 착용 여부에 따른 다중 객체 탐지 모델을 연구하였다. 실험 결과, PC 환경에서는 YOLOv8-l이 가장 높은 정확도(mAP50-95 0.694)를 보였으나, 라즈베리파이 4 환경에서는 YOLOv8-n 모델이 720.74ms의 추론 속도를 기록하며 실시간 운영에 가장 적합한 모델로 선정되었다. 시스템 구현 결과, 관리자가 설정한 동적 위험 구역에 비작업자가 50% 이상 침범 시 즉각 경고를 전송하는 현장 맞춤형 기능을 확인하였다. 본 연구는 임베디드 환경에서의 정확도와 속도 간의 현실적인 절충안을 제시하였으며, 저비용 기기로 실용적인 스마트 안전 관리 시스템 구축이 가능함을 입증하였다.

Abstract This paper proposes an edge-device-based 'AI-based Worker Safety and Hazard Detection System' using Raspberry Pi to prevent safety accidents in manufacturing and construction sites. To overcome the limitations of low versatility and the inability to distinguish between workers and non-workers in existing systems, a 3-tier architecture consisting of 'Edge Camera, Central Server, and Web Client' was designed. Furthermore, a multi-object detection model was developed to classify 'workers' and 'non-workers' based on work vest compliance and to detect helmet usage. Experimental results showed that while YOLOv8-l achieved the highest accuracy (mAP_{50-95} of 0.694) in a PC environment, the YOLOv8-n model was selected as the most suitable for real-time operation on Raspberry Pi 4, recording an inference speed of 720.74ms. The implemented system features a site-specific hazard management function where an immediate alert is triggered when a non-worker occupies more than 50% of a dynamically set hazard zone. This study presents a realistic compromise between deep learning model accuracy and inference speed in embedded environments, demonstrating the feasibility of constructing a practical, low-cost smart safety management system.

Key Words : Artificial Intelligence, Manufacturing Site, Worker Safety, Hazard Detection, YOLOv8, YOLO-NAS

1. 서론

제조 현장 및 건설 현장에서 발생하는 안전 사고는 산업 재해의 주된 원인으로 기록되고 있다. 이러한 사고의 대부분은 안전 장비 미착용, 불량착용, 또는 안전

난간 및 작업 발판 미설치와 같은 안전설비 미흡에서 비롯되며, 이는 관련 제도의 부재보다는 현장에서의 실행력이 부족한 실정임을 시사한다.

이로 인해 발생하는 인명 피해와 막대한 경제적 손실은 작업자 안전 환경 구축의 중요성을 지속적으로

This research was supported by Changwon National University in 2025~2026.

*Department of Computer Engineering, Changwon National University

**Corresponding Author: Department of Computer Engineering, Changwon National University

(donghyeokan@changwon.ac.kr)

Received December 02, 2025

Revised December 15, 2025

Accepted December 24, 2025

부각시키고 있다.

이러한 문제를 해결하기 위한 현실적인 방안으로 현장에 CCTV를 설치하여 운용하는 사례가 증가하고 있으나, CCTV가 성공적으로 설치된 경우에도 다수의 영상 화면을 제한된 소수의 관리 인력이 실시간으로 모니터링해야 하는 물리적 한계로 인해, 돌발적인 안전 사고를 즉각적으로 인지하고 방지하는 데는 명백한 부족함이 있다.

이에 따라, CCTV 영상 데이터를 딥러닝과 같은 인공지능 모델로 자동 분석하여 안전 사고 유발 요인을 선제적으로 탐지하고 방지하는 지능형 기술이 국내외에서 활발히 연구되고 있다[1, 2]. 하지만 기존에 개발된 대부분의 기술들은 특정 제조 현장이나 공정에 고도로 맞춤화되어 개발된 경우가 많아, 상이한 작업 환경을 가진 다른 현장에 범용적으로 적용하기 어렵다는 한계가 있다. 더불어, 이러한 시스템을 중소기업이 도입하기에는 상당한 비용적 부담이 존재한다.

기술적으로도 개선의 여지가 분명하다. 예를 들어, 제조업 근로자 안전관리를 다룬 기존 연구[3]의 경우 비교적 이전 세대의 모델인 YOLOv4를 기반으로 하여 mAP 81% 수준의 성능을 보고하였으며, 이는 최신 딥러닝 아키텍처를 통한 성능 개선 연구의 필요성을 제기한다. 또한, 개방된 현장에서는 작업을 수행하는 '작업자'와 외부 방문객이나 관리자 등 '비작업자'가 혼재하며, 이들에게 요구되는 안전 장비의 기준이 상이함에도 불구하고 기존 연구들은 이를 명확히 구분하여 탐지하지 못하는 문제점을 안고 있다.

따라서 본 논문에서는 다양한 제조 현장에서 유연하고 효과적으로 적용 가능한 'AI 기반 제조현장 작업자 안전 및 위험상황 검출 시스템'을 제안한다. 본 연구의 목표는 단순히 안전모 착용 여부를 탐지하는 것을 넘어, 다각적인 접근을 통해 현장의 실질적인 안전 수준을 향상시키는 데 있다.

첫째, 작업복 착용 여부를 기준으로 작업자와 비작업자를 선별하고, 각 분류의 특성에 맞게 안전모, 안전화 등 필수 안전 장비의 착용 여부를 탐지하는 다중 객체 탐지 모델을 개발한다. 이를 위해 YOLOv8과 YOLO-NAS를 비롯한 최신 객체 탐지 알고리즘들의 성능을 비교 분석하여 최적의 모델 구조를 설계한다.

둘째, 업종 간, 기업 간 상이한 작업 환경의 다양성을 고려하여, 관리자가 직접 위험 구역을 설정할 수 있는 '작업 공간(Area) 기반 좌표 매핑' 기능을 구현한다. 이는 설정된 가상의 라인이나 공간을 기준으로 작업자의 진입 및 위험 동선을 검출하는 모델을 포함하며, 현장 맞춤형 위험 관리를 가능하게 한다.

셋째, 탐지된 위험 상황(안전 장비 미착용, 위험 동선 진입)을 실시간 스트리밍 및 이벤트 알림으로 관리자가 지정한 단말에서 즉각 확인할 수 있는 모니터링 시스템을 구축한다. 이는 작업자 분류 및 안전모 미착용 검출 모델은 제시했으나, 검출 결과를 실시간으로 전송하고 알리는 모니터링 시스템 개발은 진행하지 않았던 기존 연구[4]와의 명백한 차별점이다.

마지막으로, 작업 환경이 수시로 변하거나 자원이 한정된 임시 작업장에서의 운영을 위해, 라즈베리파이(Raspberry Pi) 등을 활용한 임베디드 시스템 실험 환경을 구축하고, 기개발된 YOLOv8 및 YOLO-NAS 기반 모델들의 경량화 성능을 검증한다. 이는 PC 환경에서 제한된 이미지 데이터셋으로만 성능을 확인한 기존 연구[5]와 달리, 실제 임베디드 보드 환경에서 실시간 영상 데이터를 기반으로 모델의 실용성과 이동성을 검증한다는 점에서 차별화된다.

본 논문의 구성은 다음과 같다. 본문에서는 먼저 2장에서 시스템에 적용할 후보 모델(YOLOv8, YOLO-NAS)을 정의하고, 3장에서 학습 및 평가에 사용될 데이터셋의 구축 과정을 설명한다. 4장에서는 상세한 실험 환경 하에 모델별 학습 결과와 임베디드 환경 추론 속도를 비교 분석하여 엣지 디바이스에 탑재할 최종 모델을 선정한다. 5장에서는 엣지 카메라, 서버, 클라이언트로 구성된 3-Tier 시스템 아키텍처를 상세히 설계하며, 6장에서는 이를 기반으로 실제 구현된 시스템의 작동 결과를 보인다. 마지막으로 결론에서는 본 연구의 의의를 요약하고 향후 연구 방향을 제시한다.

2. 제안 모델

본 연구에서 제안하는 시스템은 CCTV 영상을 실시간으로 분석하여 작업자와 비작업자를 분류하고, 이들의 안전 장비(안전모, 작업복 등) 착용 여부 및 위험 구

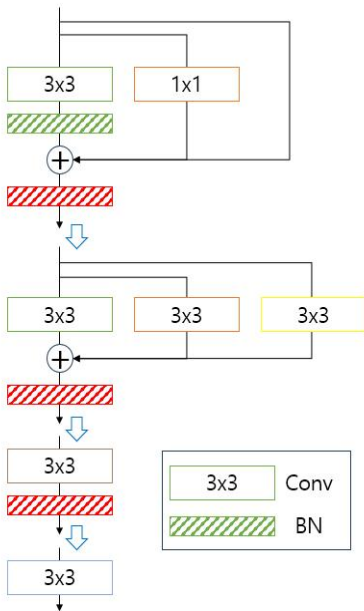


그림 1. QARepVGG Block
Fig. 1. QARepVGG Block

역 내 위험 동선을 탐지하는 것을 목표로 한다. 이러한 복합적인 작업을 고성능으로 처리하기 위해, 본 연구에서는 최신 딥러닝 객체 탐지 알고리즘인 YOLOv8[6]과 YOLO-NAS[7] 모델을 선정하여 비교 분석하였다.

두 모델은 각각 실시간 객체 탐지 분야와 경량화 및 효율화 분야에서 최고 수준의 성능을 대표하며, 특히 본 연구의 최종 목표인 임베디드 시스템(라즈베리파이)에서의 실시간 구동 가능성을 검증하는 데 적합하다.

먼저, YOLOv8은 YOLO(You Only Look Once) 시리즈의 최신 모델로, 1-stage 객체 탐지 분야에서 가장 널리 사용되고 있다. YOLOv8은 이전 버전들의 장점을 계승하면서 구조적 개선을 통해 성능을 극대화하였다. 주요 특징으로는 Anchor-Free 방식을 채택하여 복잡한 Anchor Box 설정을 제거하고 탐지 정확도와 속도를 향상시켰으며, 백본 네트워크에는 C2f 모듈을 도입하여 기존 C3 모듈보다 풍부한 그래디언트 흐름과 특징 추출 능력을 제공한다. 또한, Decoupled-Head 구조를 사용하여 분류와 박스 회귀 작업을 분리함으로써 모델의 수렴 속도와 최종 정확도를 높였다. 본 연구에서는 YOLOv8의 높은 정확도와 유연한 확장

성(n, s, m, l, x 등)을 바탕으로, 임베디드 환경에서의 고성능 탐지 모델 베이스라인으로 활용하였다.

다음으로, YOLO-NAS는 하드웨어의 특성을 고려하여 최적의 모델 아키텍처를 탐색하는 NAS(Neural Architecture Search) 기술을 기반으로 개발된 모델이다. 특히 자원이 제한된 엣지 디바이스 및 임베디드 시스템 환경에서 기존 모델들의 정확도-속도 간의 트레이드오프(trade-off) 한계를 극복하는 것을 목표로 한다. YOLO-NAS의 핵심은 'Quantization-Aware' NAS를 통해 설계된 새로운 기본 블록에 있으며, 그림 1은 이 핵심 블록 구조를 나타낸다. 이 모델은 RepVG G와 유사하게 학습 시에는 복잡한 구조를 가지지만, 추론 시에는 간단한 연산(e.g., 3x3 Conv)으로 재구성되어 추론 속도를 비약적으로 향상시킨다. 또한, NAS 과정에서부터 양자화(quantization)를 고려하여, FP32(32비트 부동소수점) 모델을 INT8(8비트 정수) 모델로 변환할 때 발생하는 정확도 손실을 최소화한다. 이는 라즈베리파이와 같이 부동소수점 연산 성능이 낮은 임베디드 장비에 배포할 때 결정적인 이점이 된다.

3. 데이터 수집 및 전처리

본 연구에서 제안하는 모델을 학습시키기 위해, 실제 제조 현장 데이터와 유사한 환경의 공개 데이터셋을 활용하였다. 이에 본 연구에서는 Kaggle[8] 및 Roboflow[9]와 같은 공개 데이터 플랫폼에서 이미지 데이터를 수집하였다.

초기 수집된 데이터는 라벨링 기준이 상이하거나 본 연구의 목적과 불필요한 클래스를 포함하고 있어, 총 531장의 이미지를 선별한 후 라벨링 정제 및 추가 작업을 수행하였다. 본 연구의 핵심 목표인 '작업자와 비작

표 1. 최종 데이터셋 클래스
Table 1. Final Dataset Class

Class	Definition	Target Object
Worker	Wearing work vest	Person
Non-Worker	Not wearing work vest	Person
Helmet-on	Wearing helmet	Head
Helmet-off	Not wearing helmet	Head

업자 구분' 및 '안전모 착용 탐지'를 동시에 수행하기 위해, 최종 데이터셋의 클래스를 표 1과 같이 총 4개로 재정의하였다.

그림 2는 실제 라벨링 예시를 보여준다. Worker와 Non-worker 클래스는 작업 조끼(vest) 착용 여부를 기준으로 사람(person) 객체 전체를 라벨링하였으며, Helmet-on과 Helmet-off 클래스는 안전모 착용 상



그림 2. 데이터 라벨링 예시
Fig. 2. Example of Data Labeling

태에 따라 머리(head) 영역을 기준으로 라벨링을 수행하였다.

수집된 원본 이미지 531장은 4개의 클래스를 가진 딥러닝 모델을 학습시키기에 부족하다고 판단하여 데이터의 양적 확보를 위해 데이터 증강(Data Augmentation) 기법을 적용하였다. 모델 학습 및 검증을 수행하기 위해 우선 수집된 데이터를 7:3의 비율로 훈련(Training) 데이터와 검증(Validation) 데이터로 분리하였다. 이후 훈련 데이터셋에 한하여, 각 이미지에 대하여 90도, 180도, 270도로 회전(Rotation) 처리를 수행하는 증강 기법을 적용하였다. 또한, 제안된 YOLO 모델들의 입력 크기에 맞추기 위해 모든 이미지의 해상도를 640x640 픽셀 크기로 일괄 변경하는 전처리를 수행하였다. 전처리 및 증강 과정을 통해 최종적으로 1484개의 학습 데이터와 160개의 검증 데이터를 구축하였다.

4. 학습 및 성능 평가

4.1 실험 환경 및 평가 지표

표 2. 상세 실험 환경

Table 2. Detailed Experiment Environment

Category	Specifications	
Model Training & Evaluation	CPU	Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz
	GPU	NVIDIA Quadro RTX 6000
	RAM	251 GiB
Inference Speed Evaluation	Raspberry Pi 4 Model B	

본 연구의 모델 학습 및 성능 평가는 표 2와 같은 환경에서 수행하였다. 학습(Training)은 고성능 GPU가 탑재된 PC 환경에서 진행하였으며, 임베디드 환경에서의 추론 속도(Inference Speed) 평가는 라즈베리 파이 4(Raspberry Pi 4)에서 수행하였다.

모델의 객체 탐지 성능은 업계 표준 지표를 기준으로 정량적으로 평가하였다. Precision(정밀도)은 모델이 Positive로 예측한 결과 중 실제 Positive인 비율($TP / (TP + FP)$)을 나타내며, Recall(재현율)은 실제 Positive인 객체 중 모델이 Positive로 예측한 비율($TP / (TP + FN)$)을 의미한다. F1 Score는 정밀도와 재현율의 조화 평균으로, 클래스 간 데이터 불균형이 존재할 때 유용한 지표이다. 또한, mAP50은 IoU(Intersection over Union) 임계값을 0.5로 고정했을 때의 평균 AP(Average Precision)로, 객체의 존재 유무 탐지 성능을 주로 평가한다. 마지막으로, mAP50-95는 IoU 임계값을 0.5에서 0.95까지 0.05 단위로 변화시키며 측정한 mAP의 평균값으로, 모델이 얼마나 정확한 위치에 바운딩 박스(Bounding box)를 예측했는지를 가장 엄격하게 평가하는 지표이다.

4.2 모델 학습 결과 및 분석

본 연구에서 구축한 데이터셋을 사용하여 YOLOv8과 YOLO-NAS의 다양한 크기(n, s, m, l, x) 모델을 학습시켰다. 표 3은 각 모델의 검증 데이터셋(Validation set)에 대한 성능 평가 결과를 요약한 것이다. 학습 결과, PC(GPU) 환경에서의 최고 성능은 모델의 크기가 큰 Yolo_v8_l과 Yolo_nas_l에서 나타났다.

표 3. 상세 실험 환경

Table 3. Detailed Experiment Environment

Model	Precision	Recall	mAP50	mAP50-95	F1	Parameter (M)
Yolo_v8_n	0.897	0.725	0.821	0.611	0.800	3.00
Yolo_v8_s	0.824	0.805	0.841	0.662	0.810	11.12
Yolo_v8_m	0.877	0.769	0.848	0.678	0.820	25.84
Yolo_v8_l	0.894	0.760	0.852	0.694	0.820	43.60
Yolo_v8_x	0.876	0.785	0.850	0.691	0.830	68.12
Yolo_nas_s	0.934	0.801	0.835	0.562	0.840	19.02
Yolo_nas_m	0.889	0.813	0.843	0.606	0.848	51.13
Yolo_nas_l	0.911	0.830	0.859	0.619	0.863	66.91

엄격한 평가 지표인 mAP50-95를 기준으로 보면, Yolo_v8_l 모델이 0.694로 모든 모델 중 가장 높은 성능을 기록했다. 이는 Yolo_v8_x (0.691)보다도 근소하게 높은 수치로, YOLOv8 계열이 YOLO-NAS 계열(최고 0.619)보다 바운딩 박스의 위치를 더 정확하게 예측하는 경향을 보였다.

반면, 객체 존재 유무 탐지 성능에 가까운 mAP50 지표에서는 Yolo_nas_l 모델이 0.859로 가장 높은 수치를 보였으며, 이는 Yolo_v8_l (0.852)보다 약간 우수했다. 또한 F1 Score 기준으로는 YOLO-NAS 계열이 전반적으로 YOLOv8 계열보다 높은 수치를 보였는데, 이는 YOLO-NAS가 객체를 탐지하는 정밀도(Precision)와 재현율(Recall)의 조화가 더 우수함을 시사한다.

4.3 임베디드 환경 추론 속도 평가

본 연구의 주요 목표 중 하나인 임시 작업장 및 이동식 환경에서의 적용 가능성을 검증하기 위해, 학습된 모델들을 라즈베리파이 4 환경에 배포하여 이미지 1장당 추론 속도(ms)를 측정하였다. 그 결과는 표 4와 같다.

측정 결과, 임베디드 환경에서는 모델의 파라미터 크기가 추론 속도에 결정적인 영향을 미치는 것을 확인하였다. 가장 파라미터 수가 적은(3.00M) Yolo_v8_n 모델이 720.74ms (약 1.39 FPS)로 가장 빠른 추론 속도를 보였다. 반면, 옛지 디바이스에 최적화된 것으로 알려진 YOLO-NAS 모델은 Yolo_nas_s

조차 1603.88ms (약 0.62 FPS)로, Yolo_v8_n보다 2배 이상 느린 속도를 기록했다. 이는 YOLO-NAS가 NPU나 특정 가속기가 없는 범용 CPU 환경(라즈베리파이)에서는 오히려 Yolo_v8_n보다 비효율적일 수 있음을 시사한다. Yolo_v8_s 이상의 모델들은 1.5초에서 10초 이상의 처리 시간이 소요되어, 실시간 모니터링 시스템에 적용하기에는 부적합한 것으로 판단되었다.

4.4 종합 분석

성능 평가 결과를 종합적으로 분석한 결과, 본 연구의 목표인 임베디드 환경에서의 실시간 운영을 위해서는 정확도와 추론 속도 간의 절충(trade-off)이 필수적임을 확인하였다. 표 3에서 보듯이, Yolo_v8_l (mAP50-95: 0.694) 및 Yolo_nas_l (mAP50: 0.859)과 같이 모델의 크기가 큰 경우 GPU 환경에서는 높은 정확도를 달성하였으나, 표 4의 라즈베리파이

표 4. 모델 별 추론 속도

Table 4. Inference Speed for Each Model

Model	Speed per image (ms)
Yolo_v8_n	720.74
Yolo_v8_s	1520.93
Yolo_v8_m	3429.66
Yolo_v8_l	6912.53
Yolo_v8_x	10208.23
Yolo_nas_s	1603.88
Yolo_nas_m	3820.28

Yolo_nas_l	5077.07
------------	---------

4 환경 테스트에서는 6초 이상의 매우 느린 추론 속도를 기록하여 실시간 적용이 불가능한 것으로 판명되었다.

반면, Yolo_v8_n 모델은 mAP50-95 기준 0.611로 대형 모델 대비 정확도는 낮았으나, 720.74ms (약 1.39 FPS)의 추론 속도를 기록하며 본 실험에 사용된 모든 모델 중 라즈베리파이 환경에서 가장 빠른 속도를 보였다. YOLO-NAS 모델의 경우, Yolo_nas_s조차 Yolo_v8_n보다 2배 이상 느린 1603.88ms를 기록하여, NPU가 없는 범용 CPU 기반 임베디드 환경에서는 Yolo_v8_n이 더 효율적임을 확인하였다.

따라서 본 연구에서는 임베디드 시스템에서의 실용성 및 배포 가능성을 최우선으로 고려하여, 비록 mAP 50-95 정확도는 0.611 수준이지만 본 실험 환경에서 가장 빠른 추론 속도를 보인 Yolo_v8_n 모델을 최종 모델로 선정하였다.

5. 시스템 아키텍처

본 연구에서 제안하는 AI 기반 안전 및 위험상황 검출 시스템은 그림 3과 같이 크게 세 가지 주요 구성 요소(3-Tier)로 설계되었다. 각 요소는 (1) 현장의 영상을 분석하고 위험을 판단하는 '엣지 카메라(Edge Camera)', (2) 데이터 중계 및 통신을 총괄하는 '중앙 서버(Server)', (3) 관리자가 현장을 모니터링하고 위험 구역을 설정하는 '웹 클라이언트(Web Client)'로 구성된다.

엣지 카메라는 3.4절에서 선정한 Yolo_v8_n 모델을 탑재한 임베디드 시스템으로, 단순 영상 스트리밍을 넘

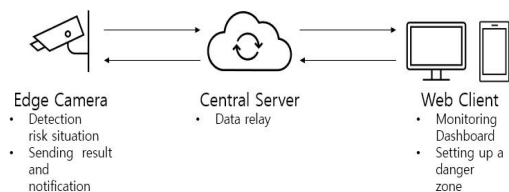


그림 3. 시스템 아키텍처

Fig. 3. System Architecture

어 현장에서 직접 딥러닝 추론을 수행한다. 1차적으로 작업자(Worker), 비작업자(Non-worker), 안전모 착용(Helmet-on), 안전모 미착용(Helmet-off) 객체를 탐지한다. 또한, 중앙 서버로부터 관리자가 설정한 '위험 구역' 좌표를 수신받아, 이 구역과 탐지된 객체의 위치를 비교하여 최종적인 위험 상황을 검출하고 그 결과를 서버로 전송하는 핵심 역할을 담당한다.

중앙 서버는 엣지 카메라와 다수의 웹 클라이언트 간의 모든 통신을 중재하는 브로커(Broker) 역할을 수행한다. 엣지 카메라로부터 실시간 탐지 결과(객체 정보, 위험 상황 발생 여부)를 수신하여, 해당 현장을 모니터링하는 웹 클라이언트로 즉시 전달한다. 반대로, 웹 클라이언트에서 관리자가 특정 카메라에 대해 '위험 구역'을 설정하면, 서버는 이 바운딩 박스 좌표 정보를 수신하여 해당 엣지 카메라로 정확하게 전송하는 양방향 통신 허브로 동작한다.

웹 클라이언트는 관리자를 위한 실시간 모니터링 대시보드 및 동적 설정 인터페이스이다. 관리자는 웹 브라우저를 통해 엣지 카메라의 실시간 탐지 결과를 모니터링할 수 있으며, 시스템이 위험 상황을 판단할 경우 즉각적인 시각적 경고 알림을 받는다. 본 시스템의 핵심 기능 중 하나로, 관리자는 모니터링 화면 상에서 동적으로 '위험 구역'을 설정하거나 변경할 수 있다. 이 정보는 즉시 서버로 전송되어 엣지 카메라의 위험 판단 로직에 반영된다. 이 기능은 작업 환경이나 공정이 변경되더라도 관리자가 유연하게 위험 구역을 재설정할 수 있게 하여 시스템의 범용성을 높인다.

본 시스템의 핵심 위험 검출 로직은 엣지 카메라단에서 수행되며, 이는 관리자가 설정한 '위험 구역'과 딥러닝 모델이 탐지한 '비작업자(Non-worker)' 객체의 중첩(Overlap)을 기반으로 한다. 그림 4는 이 검출 과정을 도식화한 것이다. 구체적으로, Yolo_v8_n 모델이 'Non-worker'로 탐지한 객체 바운딩 박스 면적의 50% 이상이, 관리자가 설정한 '위험 구역' 바운딩 박스 내에 포함될 경우, 시스템은 이를 '위험 상황'으로 최종 판단하고 경고 이벤트를 발생시킨다.

이러한 3-Tier 아키텍처 간의 모든 실시간 데이터 전송은 웹소켓(WebSocket) 프로토콜을 채택하였다. 웹소켓은 엣지 카메라의 탐지 결과를 클라이언트에 지

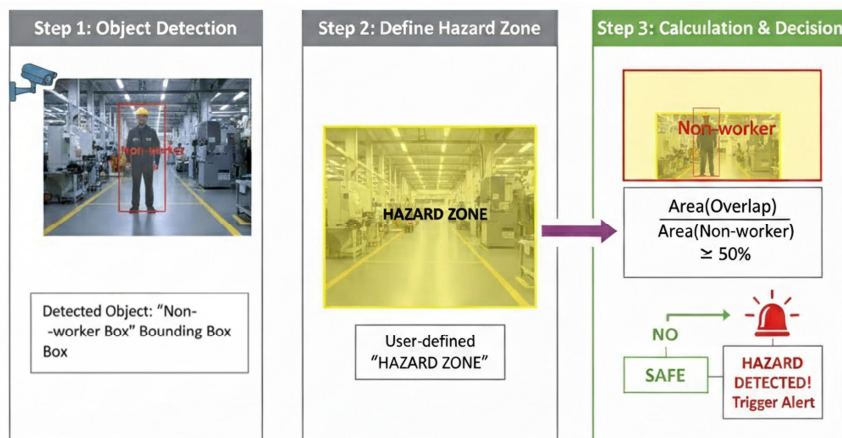


그림 4. 위험 상황 검출 과정

Fig. 4. Process of Hazard Detection

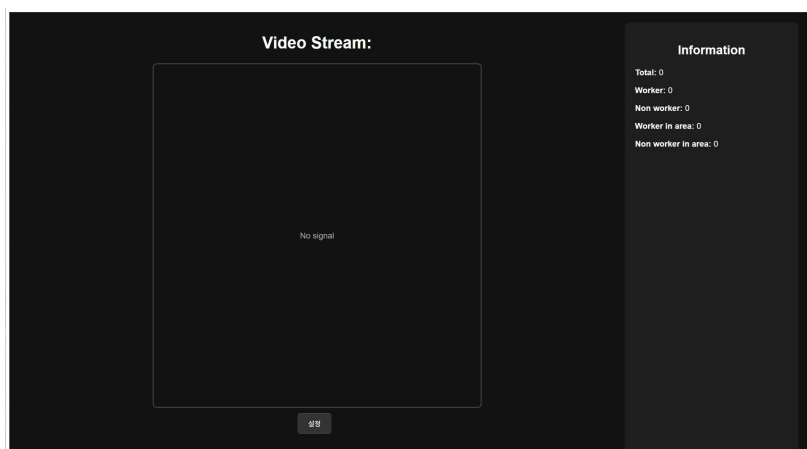


그림 5. 웹 클라이언트 화면

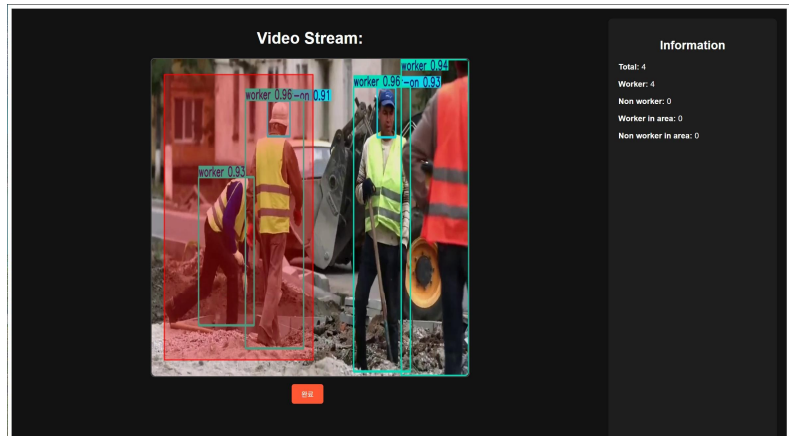
Fig. 5. Screenshot of Web Client

연 없이 전달하고, 동시에 클라이언트의 위험 구역 설정을 즉각적으로 엠티 카메라에 반영해야 하는 본 시스템의 양방향, 실시간 요구사항을 충족시키기에 가장 적합한 통신 방식이다.

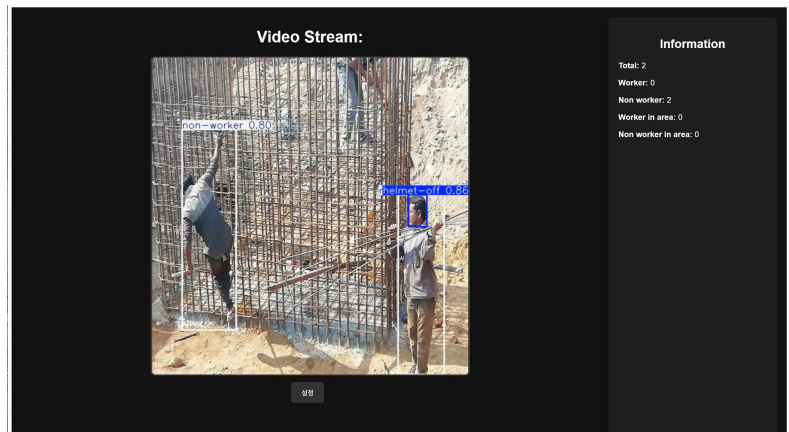
6. 시스템 구현 결과

5장에서 설계한 3-Tier 아키텍처를 기반으로 실제 안전 및 위험상황 검출 시스템을 구현하였다. 그림 5는 관리자용 웹 클라이언트의 실제 구현 화면을 보여준다.

메인 모니터링 영역 외에, 화면 우측의 정보 패널에는 엠티 카메라로부터 수신한 데이터를 실시간으로 집계하여 표시하도록 구현하였다. 여기에는 현재 탐지된 '총 인원 수', '작업자 수', '비작업자 수'와 더불어, 위험 구역 내 인원을 별도로 집계한 '위험 구역 내 작업자 수', '위험 구역 내 비작업자 수' 등의 상세 정보가 표시된다. 또한, 4장에서 정의한 '위험 구역 설정 기능'은 사용자가 메인 모니터링 화면 위에서 마우스 드래그 앤 드롭(Drag-and-Drop) 방식으로 바운딩 박스를 직접 그릴 수 있도록 구현되었다.



(a)



(b)

그림 6. 시스템 동작 결과

Fig. 6. Result of System Operation

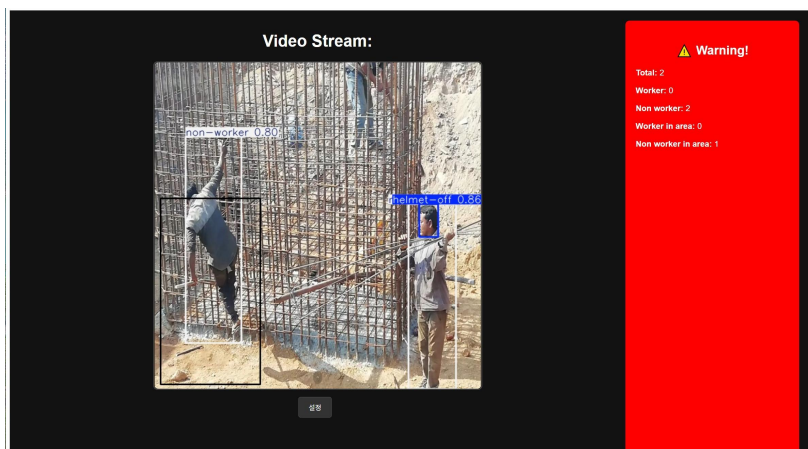
그림 6은 시스템의 실제 동작 예시를 보여준다. (a)는 관리자가 '위험 구역'을 설정하는 화면이며, 이 좌표 정보는 즉시 중앙 서버를 거쳐 엣지 카메라로 전송된다. (b)는 엣지 카메라가 Yolo_v8_n 모델을 통해 'Non-worker'(조끼 미착용) 객체를 탐지한 상태를 보여준다.

그림 7은 5장에서 설계한 핵심 위험 검출 로직이 실제로 작동하는 순간을 포착한 것이다. (a)와 (b)는 'Non-worker' 객체(흰색 박스)가 관리자가 설정한 '위험 구역'(검은색 박스)에 50% 이상 침범하자, 엣지 카메라는 이를 '위험 상황'으로 판단한다. 이 이벤트는 즉시 서버로 전송되며, 웹 클라이언트의 관리자 화면에는

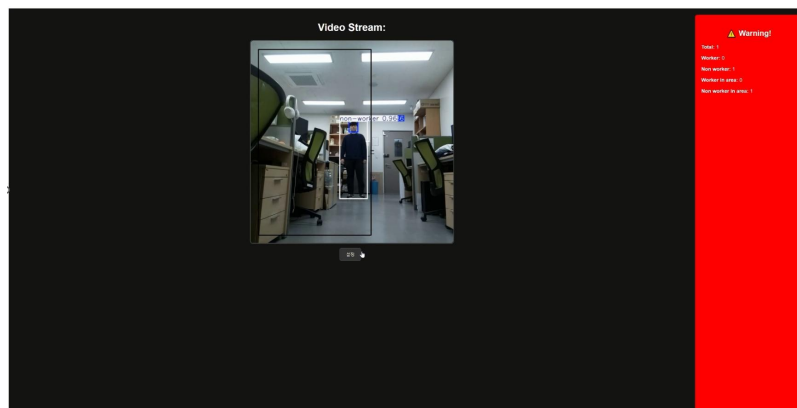
그림 7과 같이 즉각적인 '위험 경고' 알림이 발생하여 관리자가 상황을 인지할 수 있도록 하였다. 이러한 위험 검출 로직은 4장에서 검증된 모델의 mAP 0.611이라는 높은 정확도와 1.39 FPS의 안정적인 추론 속도를 바탕으로 수행되기 때문에, 실제 현장에서의 오탐을 최소화하고 즉각적인 위험 알림을 수행할 수 있다.

7. 결론

본 논문에서는 다양한 제조 현장의 안전 사고를 예방하기 위해, 엣지 컴퓨팅 기반의 AI 안전 및 위험상황



(a)



(b)

그림 7. 위험 상황 검출 결과

Fig. 7. Result of Hazard Detection

검출 시스템을 제안하고 구현하였다. 제안하는 시스템은 '엣지 카메라-중앙 서버-웹 클라이언트'의 3-Tier 아키텍처를 기반으로, 현장의 엣지 디바이스가 딥러닝 추론을 직접 수행하여 서버의 부하를 줄이고 반응 속도를 높이도록 설계되었다.

본 연구의 핵심 성과는 엣지 컴퓨팅 환경의 실용성에 초점을 맞추어, 최신 딥러닝 모델인 YOLOv8과 YOLO-NAS의 성능을 다각도로 비교 분석하고, 정확도와 추론 속도 간의 균형점을 고려하여 최종 모델(Yolo_v8_n)을 선택한 것이다. 또한, 조끼 착용 여부로 '작업자'와 '비작업자'를 구분하고, 관리자가 웹 클라이언트에서 동적으로 '위험 구역'을 설정하며, '비작업자'가 해당 구역을 50% 이상 침범 시 위험으로 판단

하는 현장 맞춤형 위험 검출 로직을 구현하였다.

향후 연구에서는 실제 제조 현장에서 촬영된 대규모 영상 데이터셋을 추가로 확보하고, 현실적인 증강 기법을 적용하여 모델의 일반화 성능을 고도화할 계획이다. 또한, 엣지 디바이스에서의 추론 속도를 30 FPS 수준의 실시간으로 개선하기 위해, 모델 양자화(Quantization)와 같은 하드웨어 가속을 위한 최적화 연구를 진행할 예정이다.

REFERENCES

- [1] S. O. Abioye et al., "Artificial intelligence in the construction industry: A review of present status, opportunities and future challenges," *Journal of Building Engineering*, vol. 44, p. 103299, Dec. 2021. (<https://doi.org/10.1016/j.jobbe.2021.103299>)
- [2] K. Lee, C. Jeon, and D. H. Shin, "Small tool image database and object detection approach for indoor construction site safety," *KSCE Journal of Civil Engineering*, vol. 27, no. 3, pp. 930-939, 2023. (<https://doi.org/10.1007/s12205-023-1011-2>)
- [3] T. Lee, Y. Kim, and H. Jung, "Dataset construction and model learning for manufacturing worker safety management," *Journal of the Korea Institute of Information and Communication Engineering*, vol. 25, no. 7, pp. 890-895, July 2021. (<https://doi.org/10.6109/jkiice.2021.25.7.890>)
- [4] Y. H. Shin et al., "Development of a construction site safety risk detection model using object detection algorithms," in *Proc. KIIE Fall Conference*, pp. 3795-3813, Nov. 2020.
- [5] W. S. Choi, J. H. Cho, W. C. Park, and S. H. Choi, "Design of YOLO-NAS model for safety helmet detection in embedded systems," *Journal of the Korea Contents Association*, vol. 24, no. 3, pp. 14-24, Mar. 2024.
- [6] Ultralytics, YOLOv8 documentation(2023), Retrieved Nov., 15, 2025, from <https://docs.ultralytics.com/models/yolov8>.
- [7] Ultralytics, YOLO-NAS documentation(2023), Retrieved Nov., 15, 2025, from <https://docs.ultralytics.com/models/yolo-nas>.
- [8] Kaggle, Kaggle datasets(2025), Retrieved Nov., 15, 2025, from <https://www.kaggle.com/datasets>.
- [9] Roboflow, Roboflow universe(2025), Retrieved Nov., 15, 2025, from <https://universe.roboflow.com>.

저자약력

김 재 영 (Jaeyoung Kim)

[학생회원]



- 2020년 3월 ~ 현재: 창원대학교 컴퓨터공학과

〈관심분야〉 네트워크 기술, 딥러닝

안 동 혁 (Donghyeok An)

[정회원]



- 2006년 2월: 한동대학교 전산전자공학부 학사
- 2013년 2월: KAIST 전산학과 박사
- 2015년 3월 ~ 2017년 8월: 계명대학교 컴퓨터공학과
- 2017년 9월~현재: 창원대학교 컴퓨터공학과

〈관심분야〉 6G, 저지연, 딥러닝